The SmartShift Challenge

Build a smart employee scheduling system

Development time: 60 minutes

Deadline: The project must be implemented within 60 minutes, so your team must finish by 15:30.

1. Background

Imagine a small company that organizes scheduled group activities. Each group requires an employee, referred to as a **host**, to welcome, guide, and supervise its activity.

The company employs several hosts, each with different availability.

Currently, the manager receives the schedule of upcoming groups in an Excel spreadsheet. Employees separately communicate their availability, and the manager manually assigns hosts to groups.

As the number of groups and employees increases, this process becomes time-consuming and difficult to manage.

Your challenge is to design and develop a digital solution that automates this process.

You may build either a **web application or a mobile/desktop application**. Your team must justify its chosen platform and explain how the design addresses the company's needs.

2. User roles

Your application must support two distinct roles.

A. Employee (Host)

Employees should be able to:

- Enter the dates and time intervals when they are available to work.
- Modify or remove their availability.
- Mark certain days as **on-call availability**: days they intentionally keep free and are willing to accept last-minute assignments.
- See their own assigned groups and working schedule.
- Record the actual number of hours worked after an assignment.
- View their own work history, including dates and hours worked.

Privacy requirement: Employees must not be able to access other employees' availability or work-hour records.

B. Manager

The manager should be able to:

- View the availability of all employees.
- See which employees have marked themselves as on-call.
- Import the upcoming group schedule from an Excel spreadsheet or by copying and pasting spreadsheet data.
- Automatically generate employee assignments.
- Review and manually modify those assignments.
- View all employees' submitted work hours.

3. The intelligent scheduling algorithm

This is the central component of the challenge.

Your application must automatically assign groups to available employees while following the rules below.

Rule 1 — Availability

Employees may only be assigned to groups that fit within their declared availability.

Assignments must not overlap in a way that requires the same employee to host more than two groups simultaneously.

Rule 2 — Fairness

Distribute assignments as evenly as reasonably possible among available employees.

An employee who is available less frequently should not automatically receive the same number of assignments as someone who is available every day.

Your team must define how fairness is measured and explain your approach.

Rule 3 — The four-hour preference

Compare the starting times of groups occurring on the same day.

- If two groups start **less than four hours apart**, preferably assign them to the **same employee**, provided that employee can accommodate both groups.
- If two groups start **more than four hours apart**, preferably assign them to **different employees**, when feasible.

These are preferences, not absolute requirements. Availability and the avoidance of overlapping assignments take priority.

For example, suppose two groups begin at 10:00 and 12:30.

Employee A is available only for the first group, while Employee B can host both.

Ideally, Employee B should receive both groups, rather than assigning one to each employee.

Rule 4 — Unassignable groups

If no employee is available for a particular group, the application must clearly identify it as **unassigned**.

Rule 5 — Manager control

The manager must be able to review the generated schedule and override an assignment if necessary.

4. Working-hours tracking

After completing an assignment, an employee should be able to submit the actual number of hours worked.

The system must store:

- Employee identity
- Date of work
- Number of hours worked

Each employee may access only their own records. The manager may access everyone's records.

The records must remain accessible during the demonstration, even after switching between employee and manager views.

5. User experience

The application should be intuitive enough that a person with no technical background can operate it without instructions.

We will assess:

- Ease of navigation
- Clarity and visual hierarchy
- Readability and accessibility
- Ease of entering or changing availability
- Clarity of the generated schedule
- Quality of error messages and feedback

A clean, well-thought-out interface is preferable to a complicated application with unnecessary features.

6. Technical expectations

You are free to choose your programming language, framework, design approach, and tools.

However, your team must be able to explain:

1. Why you chose a website or an application.
2. How your scheduling algorithm works.
3. How employee privacy is handled.
4. How fairness is measured.
5. How the system could be extended in the future.

A functional prototype is sufficient. You do not need to deploy the application publicly, integrate a real authentication service, or build production-ready infrastructure.

For demonstration purposes, simulated user accounts and locally stored data are acceptable.

7. Bonus features

Teams completing the core requirements may implement additional features.

Potential extensions include:

- **Last-minute scheduling:** Prioritize suitable on-call employees when a new group is added.
- **Schedule notifications:** Inform employees when a new assignment is created or modified.
- **Workload statistics:** Display the number of assigned groups and total hours per employee.
- **Conflict detection:** Warn the manager when a manual assignment breaks an availability rule.
- **External integration architecture:** Explain or demonstrate how the application could receive group bookings automatically from an external website
- **Better scheduling optimization:** Balance the total expected working hours rather than merely counting assignments.

8. Final demonstration

Each team will present their prototype.

During the demonstration, you will be asked to:

1. Enter or modify employee availability.
2. Import a set of group bookings.
3. Generate a schedule and explain the assignments.
4. Demonstrate employee privacy and working-hour tracking.

You should also be prepared to justify your design choices and discuss the limitations of your solution.

The goal is not to build the largest application. It is to build the most effective solution to a real scheduling problem.